FIELD NOTES

LLM API Field Guide

An LLM API bills you whether or not the answer was any good, and most of the ways it goes wrong do not raise an exception. Every script here is read only.

48 fixes, each one a written note with diagrams and a small Python and Node.js script that finds the problem and repairs it. All 48 have a script in the public repo.

Before you run anything. Every script starts in a dry run: it reports what it would change and writes nothing. Read that output first, then set `DRY_RUN=false` when you are satisfied it has understood your data.

Scripts: github.com/allanninal/llm-api-fixes
Guides: allanninal.dev/llm
All 14 repos: github.com/allanninal

The fixes

1 **an archived project still holds live API keys**

The prototype was shut down in the spring. The project was archived, which felt like closing it: it vanished from the console's project switcher and from every list anyone looks at. Nothing inside it was touched. Two keys are still enabled, one of them authenticated a request last Tuesday, and the quarterly key audit has never seen either of them, because the audit iterates projects and archived projects are not in the default listing.

`openai_archived_project_keys.py`

https://www.allanninal.dev/llm/archived-project-still-holds-keys/

https://github.com/allanninal/llm-api-fixes/tree/main/archived-project-still-holds-keys

---

2 **audio and image usage never shows up in a token dashboard**

The internal dashboard has been within a few percent of the invoice for a year, and a few percent is what everyone expects a dashboard to be. It is not rounding. It is the text-to-speech in the mobile app, the transcription on the support calls, the thumbnails the marketing tool generates, and the web search the agent does before it answers. None of those are denominated in tokens, and the endpoint the dashboard was built on only knows about tokens.

`openai_modality_spend_reconcile.py`

https://www.allanninal.dev/llm/audio-and-image-line-items-unnoticed/

https://github.com/allanninal/llm-api-fixes/tree/main/audio-and-image-line-items-unnoticed

---

3 **scheduled jobs pay full price for work the Batch API halves**

Nothing here is broken. No request failed, no row is missing, and no alert should have fired. The nightly enrichment job fires forty thousand completions between 02:00 and 02:20, finishes cleanly, and does it again the next night. It has no user waiting on it and no latency requirement of any kind, and it is being billed at the interactive rate because the synchronous endpoint is what the SDK example used. This is not a bug report. It is an invoice roughly twice the size it needs to be.

`openai_batch_discount_audit.py`

https://www.allanninal.dev/llm/batch-discount-left-unused/

https://github.com/allanninal/llm-api-fixes/tree/main/batch-discount-left-unused

---

4 **the batch left an error_file_id that nothing ever fetched**

The Batch API answers in two files. Successes go to output_file_id and failures go to error_file_id, and the ingest code was written against the first one on a day when the test batch had no failures. It has never opened the second. The failures are not lost — they were written down carefully, one JSON line each, with the custom_id and the reason. They are sitting in a file that has an id, a byte count and an expiry date, and in thirty days they will be gone whether or not anybody looked.

`openai_batch_error_file_audit.py`

https://www.allanninal.dev/llm/batch-error-file-never-read/

https://github.com/allanninal/llm-api-fixes/tree/main/batch-error-file-never-read

---

5 **a batch expired when the 24 hour completion window closed**

The submission returned 200 yesterday afternoon. The poller has been asking for the batch ever since, testing the status against completed, and it has never matched, so as far as the job is concerned the work is still running. It is not. Twenty-four hours after the batch started processing, everything OpenAI had not got to was abandoned, the status went to expired, and thirty thousand rows landed in the error file with the code batch_expired. Nothing raised, nothing retried, and the poller is still waiting.

`openai_batch_expiry_audit.py`

https://www.allanninal.dev/llm/batch-expired-past-24h-window/

https://github.com/allanninal/llm-api-fixes/tree/main/batch-expired-past-24h-window

---

6 **a batch reads completed while some of its rows failed**

The nightly job submits fifty thousand rows, sleeps, polls until the status turns to completed, downloads the output file and loads it. It has done that every night for eight months. The table it fills is short — not empty, not obviously wrong, just a few hundred rows smaller than the input file, by a number that changes every night. No exception was ever raised. The batch object says completed in plain text, and three fields further down it says "failed": 869, which nothing has ever read.

`openai_batch_partial_failure_audit.py`

https://www.allanninal.dev/llm/batch-partial-failure-unnoticed/

https://github.com/allanninal/llm-api-fixes/tree/main/batch-partial-failure-unnoticed

---

7 **Cache written on every call by a prefix that keeps moving**

Caching went in six weeks ago and the cached share never moved off zero. The obvious explanations were checked and cleared: the breakpoint is there, the prefix is long, the traffic is constant. What nobody looked at until somebody pulled the minute buckets is that the writes are not occasional. There is a write in every single minute of the window, one after another for four hours, and not one read anywhere in them. A five minute entry written at 14:03 was still alive at 14:07, and the call at 14:07 wrote a new one.

`anthropic_cache_prefix_churn.py`

https://www.allanninal.dev/llm/cache-invalidated-by-changing-prefix/

https://github.com/allanninal/llm-api-fixes/tree/main/cache-invalidated-by-changing-prefix

---

8 **cache writes are paid for and never read back**

Caching was switched on in June and the bill went up. Every call writes a fresh cache entry, is billed 1.25x base input for the privilege, and then nothing ever reads that entry back before it expires. The reason is one line: a request id got templated into the system prompt, ahead of the breakpoint, so no two prefixes have ever been byte-identical. The feature is working exactly as documented and it is costing you money.

`anthropic_cache_write_ratio.py`

https://www.allanninal.dev/llm/cache-writes-with-no-reads/

https://github.com/allanninal/llm-api-fixes/tree/main/cache-writes-with-no-reads

---

9 **code execution has spent its free 1,550 container hours**

The analytics route attaches the customer's CSV to the request, because sometimes the question is about the numbers in it and when it is, the model should be able to run something. Sometimes is about one call in forty. The other thirty-nine attach the file, spin up a container to hold it, ask something that needs no arithmetic at all, and are billed for five minutes of a machine that was never asked to do anything.

`anthropic_code_execution_hours_audit.py`

https://www.allanninal.dev/llm/code-execution-hours-exceed-free-allowance/

https://github.com/allanninal/llm-api-fixes/tree/main/code-execution-hours-exceed-free-allowance

---

10 **fast mode billed at twice the rate and served as default**

Somebody turned on Fast mode for the checkout assistant eight months ago, in the console, in an afternoon nobody wrote down. The latency graph looked better for a week. It does not look better now, and the team has spent two sprints on the retrieval step trying to work out why. The requests still carry the premium tier, the responses still return 200, and the field that says which tier actually served them is not the field anyone is logging.

`openai_fast_mode_tier_audit.py`

https://www.allanninal.dev/llm/fast-mode-silently-downgraded/

https://github.com/allanninal/llm-api-fixes/tree/main/fast-mode-silently-downgraded

---

11 **a fine-tuned model was trained, billed, and never called once**

There was a quarter when fine-tuning was going to be the answer. Four jobs were queued over three weeks, each on a slightly better training file, and the last one came back with a loss curve somebody screenshotted into Slack. Then the base model got better, or the prompt got better, or the person who cared moved teams. The model ids are still there. They still resolve. They have between them served zero requests, and the training was invoiced the month it ran.

`openai_fine_tune_usage_audit.py`

https://www.allanninal.dev/llm/fine-tuned-model-never-used/

https://github.com/allanninal/llm-api-fixes/tree/main/fine-tuned-model-never-used

---

12 **a floating model alias silently changes model under you**

No error, no deploy, no incident. The evals were 91% on Thursday and 87% on Monday, the mean output length moved, the prompt cache hit rate dropped a few points and the bill went up slightly. Everyone looks at the prompt, which did not change, and at the retrieval corpus, which did not change either. The model changed: the config names an alias, and an alias is a pointer, not a model.

`anthropic_alias_pinning_audit.py`

https://www.allanninal.dev/llm/floating-alias-instead-of-pinned-snapshot/

https://github.com/allanninal/llm-api-fixes/tree/main/floating-alias-instead-of-pinned-snapshot

---

13 **a frontier model is answering twenty-token questions**

Somebody built the intent router in an afternoon eighteen months ago. They pasted the model name out of the quickstart, because on that afternoon the question was whether the thing worked at all and the answer was worth whatever it cost. It works. It has worked every day since, four hundred thousand times a month, and every one of those calls returns a single word from a list of nine. The model that returns it is the most expensive one the organization can buy.

`openai_model_rightsizing_audit.py`

https://www.allanninal.dev/llm/frontier-model-on-trivial-workload/

https://github.com/allanninal/llm-api-fixes/tree/main/frontier-model-on-trivial-workload

---

14 **ITPM runs out because uncached input is never cached**

The 429s arrive in the afternoon and the team does what teams do: fewer workers, longer sleeps, a queue in front. None of it moves. The request counter was never the thing that ran out. What ran out was input tokens per minute, and the reason is that the same forty thousand tokens of system prompt and tool schemas are sent uncached on every single call, and every one of those tokens is charged against the limiter.

`anthropic_itpm_headroom.py`

https://www.allanninal.dev/llm/itpm-exhausted-uncached-input/

https://github.com/allanninal/llm-api-fixes/tree/main/itpm-exhausted-uncached-input

---

15 **keys still work after their owner loses project access**

She left in March. The laptop came back, SSO was switched off the same afternoon, and the offboarding ticket was closed with every box ticked. The API key she minted in her second week is still in the environment of a nightly job, still authenticating, still billing. Nothing revoked it, because nothing was ever asked to: the key is a separate object from her membership, and removing the membership left the key exactly as it was.

`openai_orphaned_key_audit.py`

https://www.allanninal.dev/llm/key-owner-lost-project-access/

https://github.com/allanninal/llm-api-fixes/tree/main/key-owner-lost-project-access

---

16 **A live project's usage buckets have been empty for days**

A customer asks, politely, when the summaries are coming back. Nobody on the call knows what they mean, because the feature works: the page renders, the job runs, the queue is empty, the error rate is zero and the latency graph is the flattest it has been all year. It is flat because for eleven days that project has sent the API nothing at all. A feature flag went the wrong way in an unrelated release, and every dashboard the team owns measures things that only exist when requests do.

`openai_project_went_quiet.py`

https://www.allanninal.dev/llm/live-project-zero-usage-buckets/

https://github.com/allanninal/llm-api-fixes/tree/main/live-project-zero-usage-buckets

---

17 **most of your input tokens sit in the 200k-1M band**

The agent keeps everything. That was the design: give it the whole ticket history, the whole document, every tool result it has ever produced in this session, and let it decide what matters. It worked beautifully in testing, where a session was four turns. In production a session is forty, each turn carries everything the previous thirty-nine produced, and the prefix has quietly grown to four hundred thousand tokens that get sent again from scratch every single time somebody types.

`anthropic_long_context_audit.py`

https://www.allanninal.dev/llm/long-context-requests-unwatched/

https://github.com/allanninal/llm-api-fixes/tree/main/long-context-requests-unwatched

---

18 **max_tokens is set above the model's own output cap**

The classifier was moved onto the cheap model on a Friday, which is the correct thing to do with a classifier. It is the same helper function everything else uses, so it inherited the same max_tokens, which is a number somebody picked for the model that writes reports. Every call to it now comes back 400, and the message says exactly what is wrong, and the message is in a log that no dashboard reads.

`anthropic_max_tokens_cap.py`

https://www.allanninal.dev/llm/max-tokens-above-model-cap/

https://github.com/allanninal/llm-api-fixes/tree/main/max-tokens-above-model-cap

---

19 **a model id in use is past its published shutdown date**

Nothing was deployed. The key did not rotate. One model id started returning 404 on the same morning for every request, and the message reads exactly like a typo: The model does not exist or you do not have access to it. It existed yesterday. There is no distinct error code for a retired model, no deprecation warning on the successful calls that came before it, and nothing in the response that tells the difference between a model that was shut down and a model name somebody misspelled.

`openai_model_shutdown_audit.py`

https://www.allanninal.dev/llm/model-past-shutdown-date/

https://github.com/allanninal/llm-api-fixes/tree/main/model-past-shutdown-date

---

20 **a model you still call retires in under 90 days**

Every call is returning 200. Latency is normal, cost is normal, the evals are green. The only thing wrong is a field nobody is reading: shutdown_date on the model you route most of your traffic to is a real date, and it is close. Nothing will change until that morning, and then everything will, all at once, in every code path that names the id.

`openai_model_retirement_window.py`

https://www.allanninal.dev/llm/model-retiring-within-90-days/

https://github.com/allanninal/llm-api-fixes/tree/main/model-retiring-within-90-days

---

21 **no hard spend limit is set, so the bill has no ceiling**

The console shows a budget chart, and everybody who has looked at it believes it is a brake. It is not; it is a chart. The hard limit is a separate object on a separate admin endpoint, it is off by default, and until somebody turns it on there is no amount of spend in a month that will cause a request to be refused.

`openai_spend_limit_audit.py`

https://www.allanninal.dev/llm/no-organization-spend-limit/

https://github.com/allanninal/llm-api-fixes/tree/main/no-organization-spend-limit

---

22 **A non-streaming request over 10 minutes times out with 504**

The prompt is two thousand tokens. The context window is nowhere near full. Nothing is too large by any measure anybody in the room has checked, and the request still dies — sometimes as 504 with timeout_error, more often as nothing at all, because a load balancer somewhere between you and Anthropic closed an idle connection while the model was still writing. The ceiling this hit is a clock.

`anthropic_wall_clock_preflight.py`

https://www.allanninal.dev/llm/non-streaming-request-over-ten-minutes/

https://github.com/allanninal/llm-api-fixes/tree/main/non-streaming-request-over-ten-minutes

---

23 **one line item or project is most of the organization's bill**

The bill is roughly what everyone expected, which is why nobody has opened it. When somebody finally does, one line item is seventy-eight percent of it, and it is not the one the team spent last quarter optimising. There is no error here and nothing failed. There is a month of engineering time that went into a row worth three percent of the total, because the report was read as a single number and single numbers do not have a shape.

`openai_cost_concentration_audit.py`

https://www.allanninal.dev/llm/one-model-or-project-dominates-cost/

https://github.com/allanninal/llm-api-fixes/tree/main/one-model-or-project-dominates-cost

---

24 **output tokens per minute is the real ceiling, not RPM**

The capacity plan is a spreadsheet with requests per minute in it, and it has been right about everything for a year. Then thinking gets turned up on the summariser, and the same number of calls starts 429ing. Nobody changed the request rate. Nobody changed the prompts. What changed is that each answer got four times longer, and the limiter that was never in the spreadsheet is the one counting characters as they come out.

`anthropic_otpm_ceiling.py`

https://www.allanninal.dev/llm/otpm-exhausted/

https://github.com/allanninal/llm-api-fixes/tree/main/otpm-exhausted

---

25 **output tokens, not input, are what the bill is made of**

Every conversation about LLM cost starts with the prompt. The system prompt gets trimmed, the few-shot examples get cut, someone measures the context window. Then you group the cost report by token_type and find that three-quarters of the money is on the other side of the request, where none of the levers you just pulled reach.

`anthropic_output_cost_audit.py`

https://www.allanninal.dev/llm/output-tokens-dominate-cost/

https://github.com/allanninal/llm-api-fixes/tree/main/output-tokens-dominate-cost

---

26 **529 overloaded errors arrive in clusters and get dropped**

Three minutes on a Tuesday afternoon, and about fourteen hundred jobs are simply not in the output table. Nothing crashed. The worker's error counter shows a spike of something it logged as unexpected status, because the client special-cases 429 and 500 and lets everything else fall through to a generic failure path that drops the work. The status was 529, the platform was over capacity for four minutes, and the only trace left is that Anthropic did no work in those minutes while your client believed it was sending plenty.

`anthropic_overload_residual.py`

https://www.allanninal.dev/llm/overloaded-529-clusters/

https://github.com/allanninal/llm-api-fixes/tree/main/overloaded-529-clusters

---

27 **Parallel tool calls void the strict schema guarantee**

The schemas are strict, every one of them, because somebody read the Structured Outputs page properly and did the work. The parser has no try/except around it, deliberately: the arguments are guaranteed to conform, so a failure there should be loud. It has been loud four times in three months, always on a Tuesday afternoon, always with a stack trace that says a required field was missing, and always unreproducible from the same prompt. The four turns have one thing in common that nobody looked at: each of them called two tools instead of one.

`openai_parallel_strict_calls.py`

https://www.allanninal.dev/llm/parallel-tool-calls-with-strict-schema/

https://github.com/allanninal/llm-api-fixes/tree/main/parallel-tool-calls-with-strict-schema

---

28 **per-customer cost is unknowable because tenants share a key**

Finance asks what the largest account costs to serve. It is a reasonable question and somebody says it will take an afternoon, because the Usage API has a group_by=user_id and the application has been sending a user field on every request since the first week. The afternoon produces a table with eleven rows in it. Nine are engineers, two are service accounts, and not one of them is a customer.

`openai_tenant_attribution_audit.py`

https://www.allanninal.dev/llm/per-tenant-cost-attribution-impossible/

https://github.com/allanninal/llm-api-fixes/tree/main/per-tenant-cost-attribution-impossible

---

29 **prompt caching was never switched on anywhere**

The system prompt is four thousand tokens of instructions, a tool catalogue and two worked examples. It is identical on every call, and there are three hundred thousand calls a month. It has been reprocessed at the full input rate every single time, because prompt caching is opt-in and nobody opted in. There is no error to find, no warning header, and no line in the invoice that says what this cost. There is only a field in the usage report that has been zero since the day the integration shipped.

`anthropic_prompt_cache_off.py`

https://www.allanninal.dev/llm/prompt-caching-never-used/

https://github.com/allanninal/llm-api-fixes/tree/main/prompt-caching-never-used

---

30 **Prompts overflow the context window and 400 as too long**

The retrieval step got better last quarter, so it returns eight chunks instead of five. The agent loop got longer, because agents do. The tool list grew by four definitions nobody costed. None of those three changes touched the prompt template, and none of them was reviewed as a capacity change, and one afternoon the request that has worked for a year comes back 400 with prompt is too long.

`anthropic_context_preflight.py`

https://www.allanninal.dev/llm/prompt-too-long-context-overflow/

https://github.com/allanninal/llm-api-fixes/tree/main/prompt-too-long-context-overflow

---

31 **429 credit_balance_exhausted retried forever as a rate limit**

Traffic did not degrade, it stopped. Every request comes back 429, your retry wrapper does what it was built to do, and eight hours later it is still doing it. The status code says slow down. The code field inside the body says the account is out of money, and nothing in the SDK draws a line between the two — RateLimitError is raised for both.

`openai_quota_wall_audit.py`

https://www.allanninal.dev/llm/quota-exhausted-not-rate-limited/

https://github.com/allanninal/llm-api-fixes/tree/main/quota-exhausted-not-rate-limited

---

32 **429s are retried blindly without reading which limit hit**

The handler is four lines long and it has been in production for a year. Catch the 429, sleep, retry, give up after three. It works, in the sense that the process does not crash. It has also never once told anybody which of three separate limiters emptied, because the answer was in the response headers and the handler caught an exception class instead of reading a response.

`anthropic_limiter_identify.py`

https://www.allanninal.dev/llm/rate-limit-429-limiter-unidentified/

https://github.com/allanninal/llm-api-fixes/tree/main/rate-limit-429-limiter-unidentified

---

33 **x-ratelimit-remaining sits near zero before any 429**

Nothing has failed. There is no incident, no 429 in the logs, no page. There is a number that says you are running on four percent of your token budget at four in the afternoon, and it arrives attached to every successful response your application has ever received, and no code you own has ever looked at it.

`openai_rate_limit_headroom.py`

https://www.allannінal.dev/llm/rate-limit-headers-near-exhaustion/

https://github.com/allannінal/llm-api-fixes/tree/main/rate-limit-headers-near-exhaustion

---

34 **Requests billed, zero output tokens: max_tokens refused**

The model constant changed in a one-line pull request, because the old id has a shutdown date and somebody diaried it properly. The deploy went out on Thursday. Nothing paged: the endpoint returns a 500 to the user and the retry wrapper swallows it, and the error-rate dashboard is scoped to the gateway rather than to this worker. What the organization usage report shows for Friday is eleven thousand requests against the new model, no input tokens, and no output tokens at all.

`openai_zero_output_buckets.py`

https://www.allannінal.dev/llm/reasoning-model-rejects-max-tokens/

https://github.com/allannінal/llm-api-fixes/tree/main/reasoning-model-rejects-max-tokens

---

35 **reasoning tokens are billed as output but never returned**

Somebody changed one model constant. The answers coming back are the same length as before, the prompts are the same length as before, the request count is flat — and the line for that model on the cost report went up by a factor of four. The tokens you are paying for were generated, billed at the output rate, and then not returned to you.

`openai_reasoning_token_audit.py`

https://www.allannінal.dev/llm/reasoning-tokens-billed-invisibly/

https://github.com/allannінal/llm-api-fixes/tree/main/reasoning-tokens-billed-invisibly

---

36 **The model refused and the refusal field was never read**

The support summariser stopped producing summaries for a particular kind of ticket. Not all of them, and not with an error: the row was written, the summary column held an empty string, and the dashboard that counts summaries counted them. It took a week and a customer complaint before anyone read a stored response end to end, and there it was, in a content item nobody's code had ever touched: the model had declined, politely, and said why.

`openai_refusal_channel.py`

https://www.allannінal.dev/llm/refusal-field-ignored/

https://github.com/allannінal/llm-api-fixes/tree/main/refusal-field-ignored

---

37 **A 32 MB request is rejected with 413 before Anthropic sees it**

The PDF is 24 MB, which everybody agreed was fine, because the context window is 200,000 tokens and a 24 MB PDF is nothing like 200,000 tokens. The request comes back 413 anyway, with a body that does not look like Anthropic's usual error envelope, and nothing about it appears in the usage report. It never reached Anthropic. It was refused by a proxy in front of the API, for a reason that has nothing to do with tokens.

`anthropic_request_bytes.py`

https://www.allannінal.dev/llm/request-too-large-413/

https://github.com/allannінal/llm-api-fixes/tree/main/request-too-large-413

---

38 **Request count tripled while token volume stayed flat**

Latency got worse over about a fortnight, in the way that nobody can date precisely. Then 429s started arriving at a volume that used to be comfortable, and the obvious explanation was growth, except that the invoice barely moved. Two numbers sit in the same usage report and they have come apart: requests are up three times and tokens are up not at all. Two thirds of the extra calls landed in seventeen hours out of a hundred and sixty-eight, which is not what a new customer looks like.

`openai_retry_storm_shape.py`

https://www.allanninal.dev/llm/requests-diverge-from-token-volume/

https://github.com/allanninal/llm-api-fixes/tree/main/requests-diverge-from-token-volume

---

39 **a retired model id still sitting in the code**

A batch job that runs on the first of the month failed on every request with 404 and "type": "not_found_error", message The requested resource could not be found. The endpoint is right, the key works, the same key runs the rest of the application all day. The model id in that job's params block was retired months ago, and nothing else in the codebase names it, so nothing else broke and nobody knew.

`anthropic_model_ids_audit.py`

https://www.allanninal.dev/llm/retired-model-id-still-in-code/

https://github.com/allanninal/llm-api-fixes/tree/main/retired-model-id-still-in-code

---

40 **spend jumped week over week and no release explains it**

The invoice is three times last month's and nothing shipped. There is no error to look up, no status code, no failed request — the API worked perfectly all month, which is the problem. Somewhere in the last six weeks a cron went from hourly to every five minutes, or a prompt template grew a retrieved document, or a customer onboarded, and the feedback loop between that change and the bill is measured in weeks. By the time the number arrives, nobody remembers the week it started in.

`llm_spend_week_over_week.py`

https://www.allanninal.dev/llm/spend-spike-week-over-week/

https://github.com/allanninal/llm-api-fixes/tree/main/spend-spike-week-over-week

---

41 **streamed responses report no usage and the dashboard undercounts**

The internal cost dashboard has been trusted for a year. It reads the usage object off every response, sums it per project, multiplies by the price card and draws a line. In January the line and the invoice were within a few percent of each other. They are not now, and the gap is a third. Nothing in the dashboard is broken, and nothing in the pipeline dropped a record: the chat endpoint was switched to streaming in March, and a streamed chunk carries usage: null.

`openai_streaming_usage_gap.py`

https://www.allanninal.dev/llm/streaming-usage-lost/

https://github.com/allanninal/llm-api-fixes/tree/main/streaming-usage-lost

---

42 **strict omitted, so the JSON schema is only a suggestion**

The validator threw about once every fifty calls, always in production, never in a test. Sometimes an extra key nobody had asked for; sometimes an optional field missing that the schema said was required; once a number arriving as a string. It read like flakiness, so it got a retry, and the retry mostly worked, which settled the matter for four months. The schema had been attached to every one of those calls. It had also never once been enforced, because somebody had taken the strict flag out eleven months earlier to make a 400 go away.

`openai_advisory_schema.py`

https://www.allanninal.dev/llm/strict-false-schema-silently-ignored/

https://github.com/allanninal/llm-api-fixes/tree/main/strict-false-schema-silently-ignored

---

43 **JSON cut off mid-object because the ceiling was reached**

The extraction pipeline had been green for six weeks and then started dropping about one document in forty into the dead-letter queue with a JSONDecodeError. The traceback pointed at a worker three hops from any HTTP client, so the first day went on the queue and the second on the worker. What it turned out to be was a 200. The model had followed the schema exactly, the way strict mode promises, and had been cut off halfway through the fourth line item of an unusually long invoice. Everything about that request succeeded, including the bill.

`openai_truncated_structured_output.py`

https://www.allanninal.dev/llm/structured-output-truncated-by-length/

https://github.com/allanninal/llm-api-fixes/tree/main/structured-output-truncated-by-length

---

44 **Tool-call arguments that parse and still break the schema**

The agent had been running for a month when a run started dying in the middle of a turn, roughly once in three hundred calls, always on the same tool. The traceback was a KeyError inside the handler, four frames below anything that knew about HTTP. The arguments string had parsed without complaint. It was well-formed JSON, it was even plausible JSON, and it described a call to a function whose signature had changed in a pull request the tool schema had not followed.

`openai_tool_call_arguments.py`

https://www.allanninal.dev/llm/tool-call-arguments-unparseable/

https://github.com/allanninal/llm-api-fixes/tree/main/tool-call-arguments-unparseable

---

45 **Tool shipped on every request and never once called**

The tool registry has twenty-six entries. Nineteen of them were written in the same fortnight by four people, and the descriptions read like function signatures because that is what they were copied from. Every one of them goes out on every request, because the registry is built once at start-up and handed to the client whole. Nothing is broken. The handlers for six of those tools have not been entered in production since the day they were merged, and the tools are still being paid for on every turn.

`openai_dead_tool_definitions.py`

https://www.allanninal.dev/llm/tool-defined-but-never-called/

https://github.com/allanninal/llm-api-fixes/tree/main/tool-defined-but-never-called

---

46 **Tool schemas are most of the input tokens on every call**

The agent has forty tools because forty things are worth doing, and each definition is a careful JSON schema with descriptions on every property because that is how you get the arguments right. A support turn is one sentence from a customer. Nobody has ever asked what the block in front of that sentence weighs, and the answer, when somebody finally counts it, is that the machinery is thirteen times the conversation on every single call.

`anthropic_tool_schema_overhead.py`

https://www.allanninal.dev/llm/tool-schemas-dominate-input-tokens/

https://github.com/allanninal/llm-api-fixes/tree/main/tool-schemas-dominate-input-tokens

---

47 **US inference geo is billing every token at 1.1x**

Somebody in a procurement call last spring said that the data has to stay in the US, and somebody else, being helpful, went and set the workspace default that afternoon. Nobody wrote it down, because it took four seconds. The contract that prompted it was signed for one customer. The workspace serves all of them, and every token any of them has generated since has been billed at one and a tenth times the rate card.

`anthropic_inference_geo_premium_audit.py`

https://www.allanninal.dev/llm/us-inference-geo-premium-unnoticed/

https://github.com/allanninal/llm-api-fixes/tree/main/us-inference-geo-premium-unnoticed

---

48 **web search is billing $10 per 1,000 searches unnoticed**

The research agent was allowed to search the web in March, because an agent that cannot look anything up is a party trick. Nobody put a ceiling on how many times it could search in one turn, because in March it searched twice. It now runs on every support ticket, it searches until it is satisfied, and satisfied averages eleven. None of that is a token, so the token dashboard on the wall behind the standup has never once flickered.

`anthropic_web_search_spend_audit.py`

https://www.allanninal.dev/llm/web-search-spend-unnoticed/

https://github.com/allanninal/llm-api-fixes/tree/main/web-search-spend-unnoticed

---